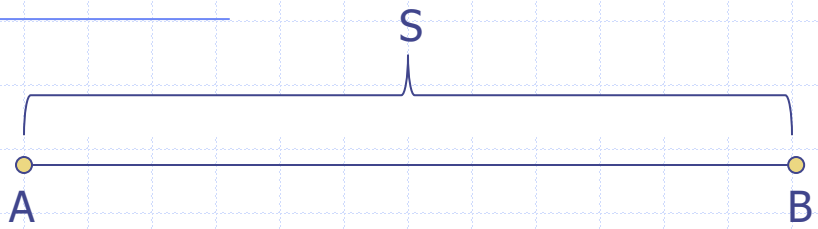
Paralelno programiranje

- Pronalaženje paralelizma
- Uobičajene strukture paralelnih algoritama

Šta utiče na performase paralelnih programa?

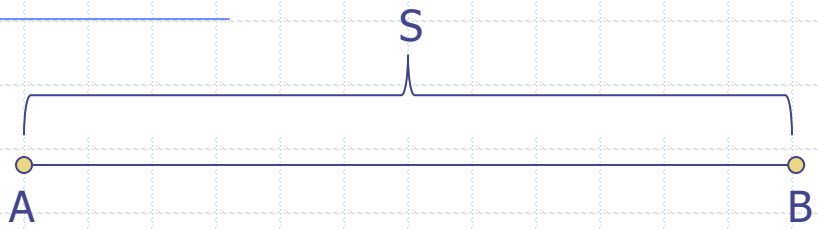
- Pokrivenost
  - koliki deo programa može da se izvršava paralelno
- Granularnost
  - količinska raspodela posla na procesore
- Lokalnost
  - Podataka i računanja

Zadatak



- Od mesta A do mesta B ima 100km.
- Ako automobil pređe prvu polovinu puta brzinom od 50km/h, kolikom brzinom treba da pređe drugu polovinu puta da bi mu prosečna brzina na celom putu bila 100km/h?

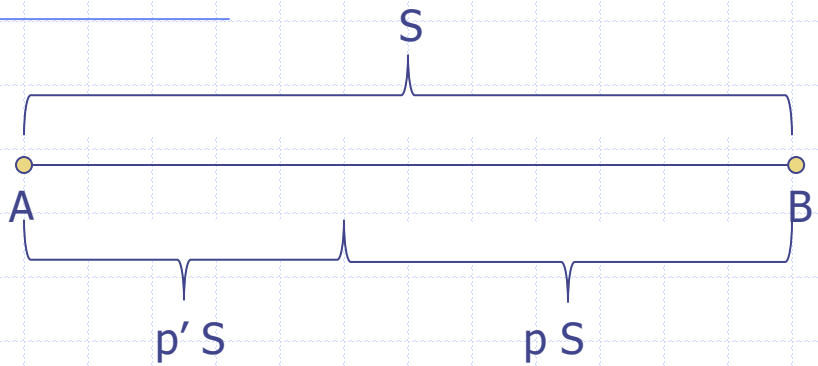
Zadatak



- Od mesta A do mesta B ima 100km.
- Ako automobil pređe prvu polovinu puta brzinom od 50km/h, kolikom brzinom treba da pređe drugu polovinu puta da bi mu prosečna brzina na celom putu bila 100km/h?

~~150km/h~~

Zadatak



v_1 - brzina na prvom delu puta

v_2 - brzina na drugom delu puta

v_s - srednja brzina na celom putu

$$p + p' = 1;\ p, p' \in \mathbb{R}^+$$

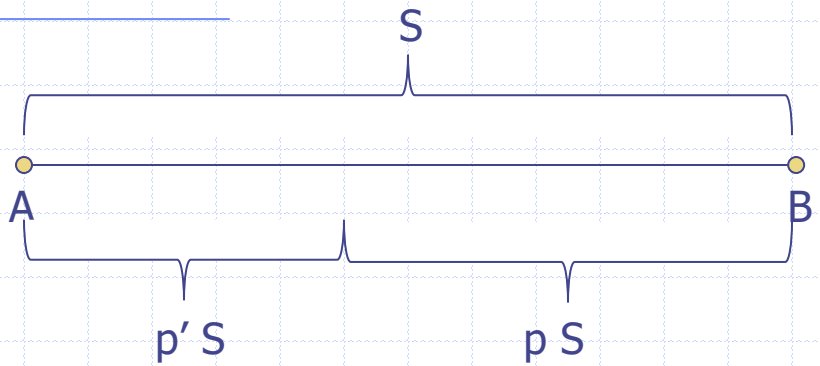
$$v_2 = xv_1 \qquad U = \frac{v_s}{v_1}$$

$$v_s = \frac{S}{t_1 + t_2} \qquad t_1 = \frac{p'S}{v_1} \qquad t_2 = \frac{pS}{xv_1}$$

$$v_s = \frac{S}{\frac{p'S}{v_1} + \frac{pS}{xv_1}} = \frac{S}{\frac{S}{v_1}\left(p' + \frac{p}{x}\right)} = \frac{v_1}{p' + \frac{p}{x}}$$

$$U = \frac{1}{p' + \frac{p}{x}} = \frac{1}{(1-p) + \frac{p}{x}}$$

Zadatak



$p = 0{,}5$

$v_1 = 50km/h$

$v_2 = xv_1$

$U = 2$

$$U = \frac{1}{(1-p) + \frac{p}{x}} = 2$$

$$(1-p) + \frac{p}{x} = 0{,}5 \qquad 0{,}5 + \frac{0{,}5}{x} = 0{,}5 \qquad 1 + \frac{1}{x} = 1$$

$$\frac{1}{x} = 0 \qquad x = \infty \qquad v_2 = \infty$$

Pokrivenost
Amdalov zakon

- Maksimalno ubrzanje koda uvođenjem paralelizma je ograničeno procentom koda koji može biti paralelizovan
- Maksimalno ubrzanje: $\frac{1}{(1-p)+\frac{p}{n}}$ gde je n broj procesora, a p procenat paralelnog koda

Granularnost

- Mera odnosa vremena koje ode na obradu i koje ode na režiju i komunikaciju

Fina granulacija:

- Mali odnos obrade i režije (više režije)
- Mala količina posla između perioda komunikacije
- Velika režija za **komunikaciju** i raspoređivanje

Gruba granulacija:

- Veliki odnos obrade i režije (više obrade)
- Velika količina posla između perioda komunikacije
- Teže **balansirati opterećenje** (više praznog hoda)

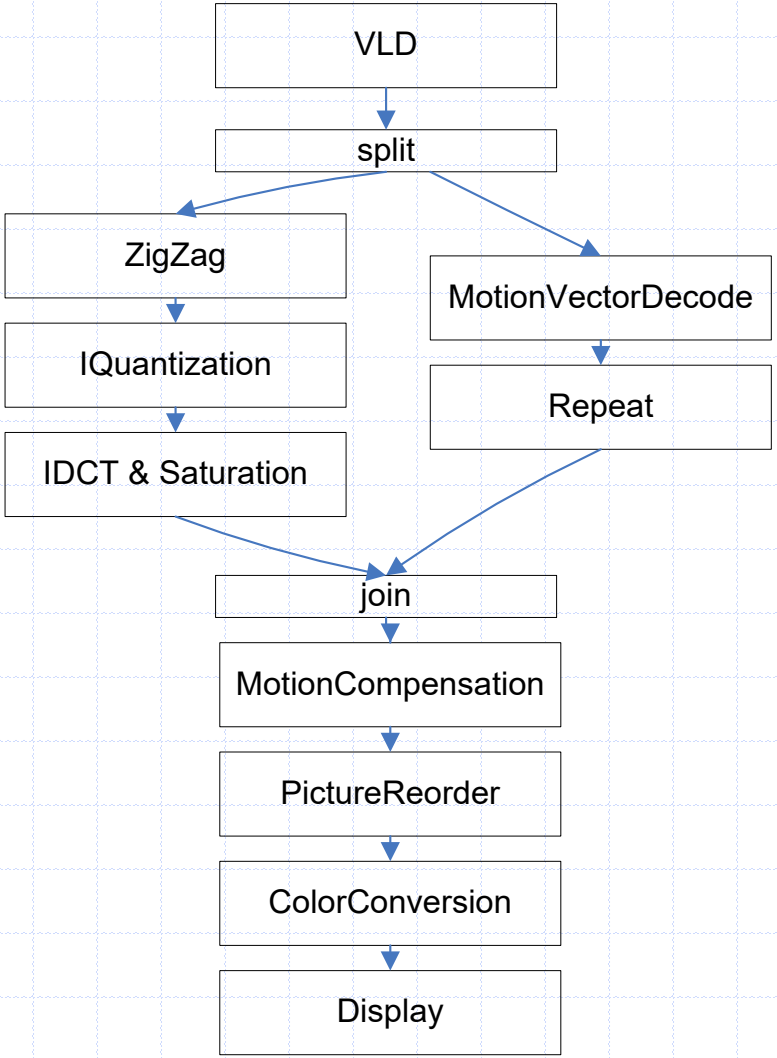
Lokalnost

- Tumačenje može da zavisi od tipa platforme na kojoj se obavlja paralelizacija (mreža računara, procesori sa deljenom memorijom, sa odvojenom memorijom, keš memorija...)
- Ali u suštini, što više lokalnosti, to manje delenja resursa i, samim tim, manje guranja za pristup resursima.

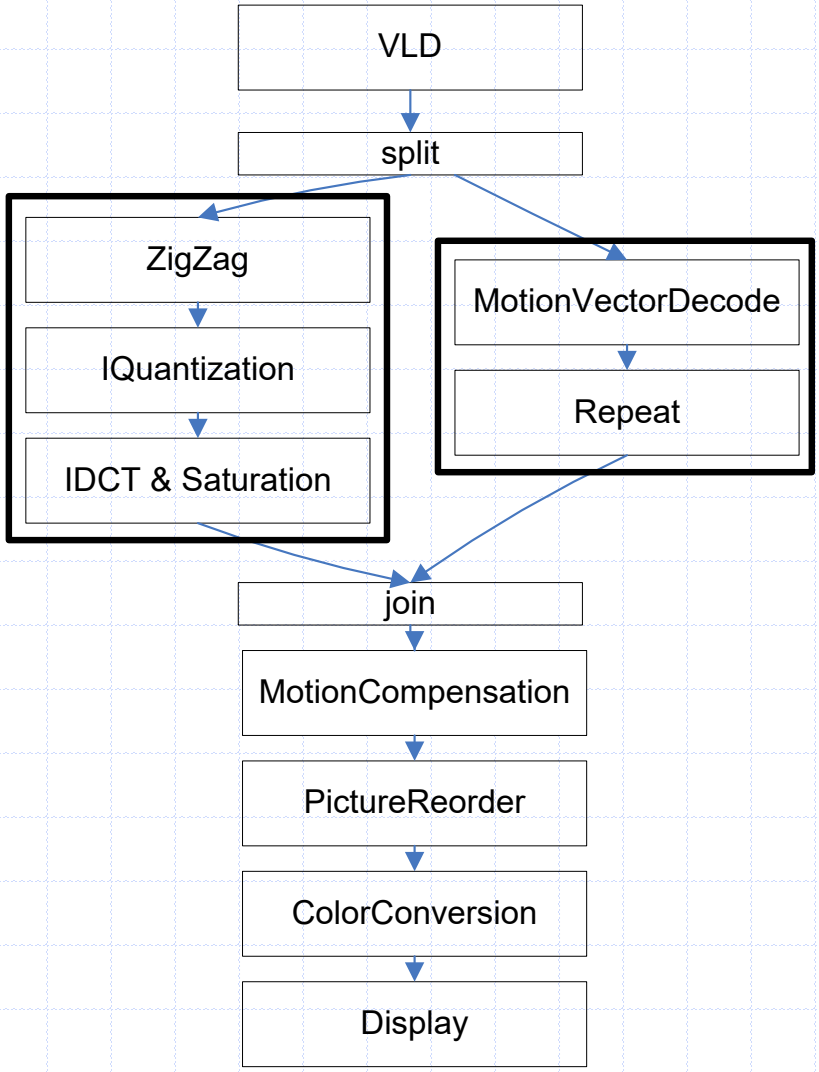
Četiri projektantska prostora

- Izražavanje algoritma
  - I Pronalaženje paralelizma
    - Uočavanje konkurentnih zadataka
  - II Struktura algoritma
    - Preslikavanje zadataka na procese radi korišćenja paralelnih arhitektura
- Konstruisanje programa
  - III Pomoćne strukture
    - Obrasci koda i struktura podataka
  - IV Izvedbeni mehanizmi
    - Mehanizmi niskog nivoa koji se koriste za pisanje paralelnih programa

Evo algoritma.
Gde je paralelizam?

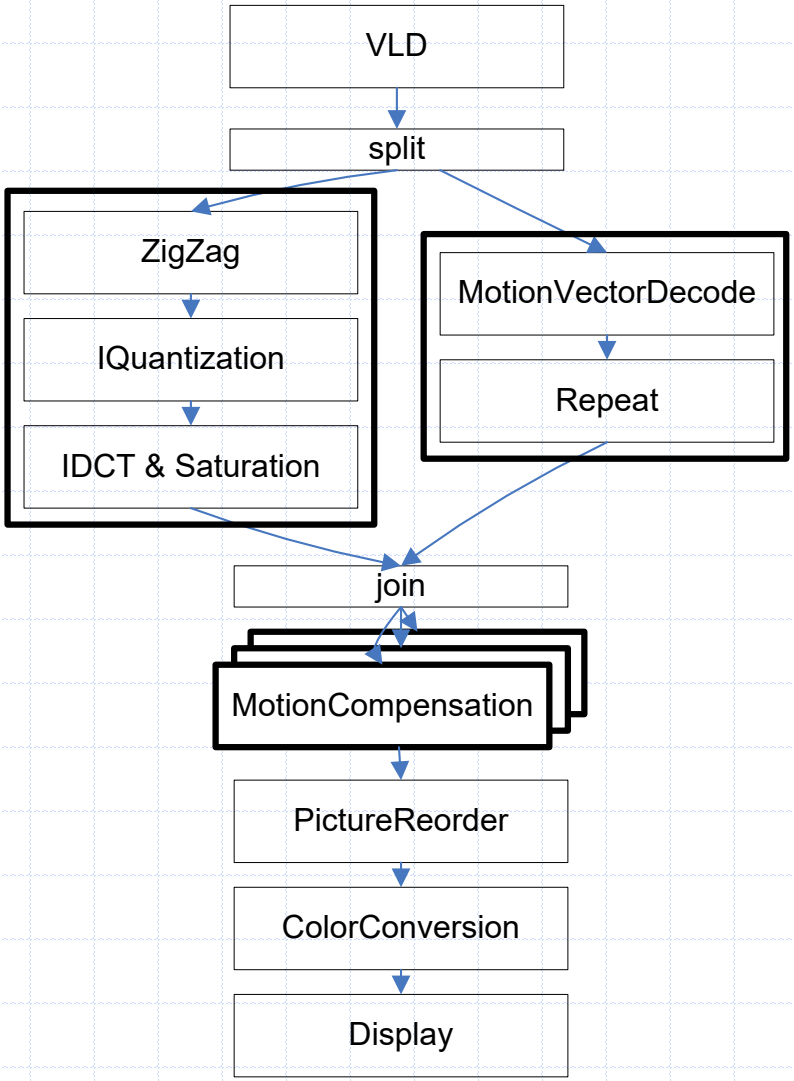


Evo algoritma.
Gde je paralelizam?



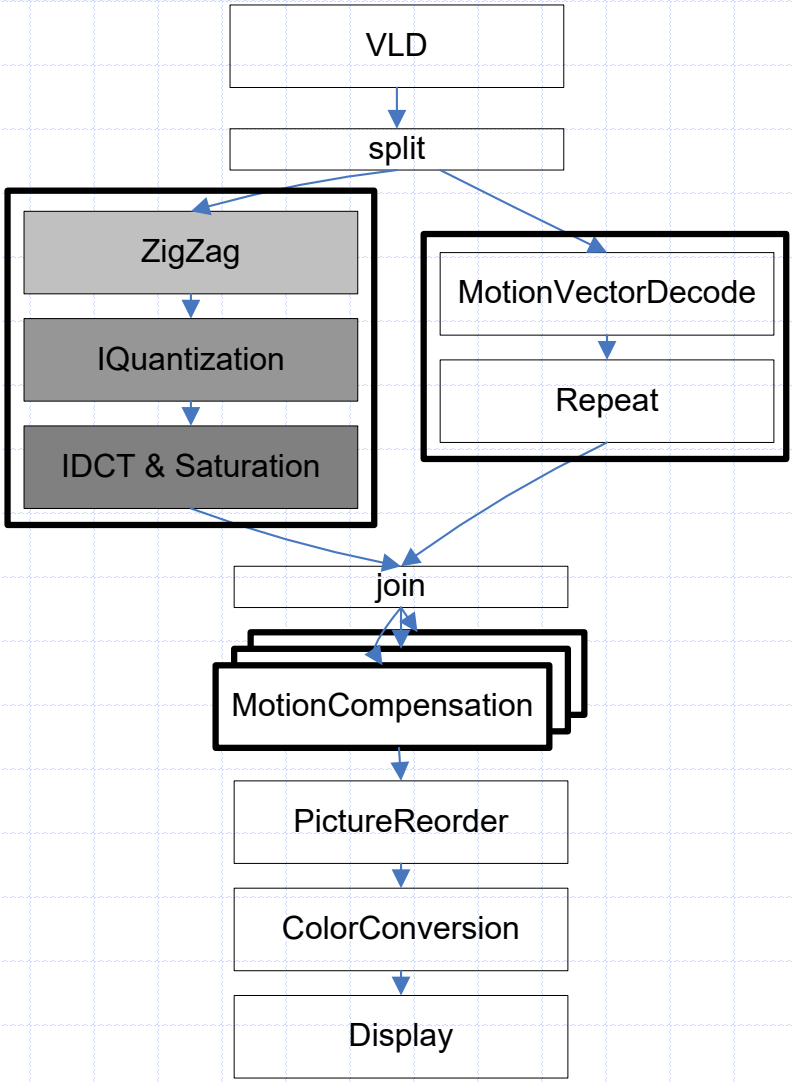
- Dekompozicija zadataka
  - Nezavsine krupnije obrade (gruba granulacija)
  - Svojstvene algoritmu
  - Niz iskaza koji operišu kao grupa (Odgovaraju nekom logičkom delu programa i obično prate način na koji programer razmišlja o problemu)

Evo algoritma.
Gde je paralelizam?



- Dekompozicija zadataka
  - Paralelizam u aplikaciji
- Dekompozicija podataka
  - Ista obrada se primenjuje na malim blokovima podatka, koji su izvedeni iz velikog skupa podataka

Evo algoritma.
Gde je paralelizam?



- Dekompozicija zadataka
  - Paralelizam u aplikaciji
- Dekompozicija podataka
  - Ista obrada, više podataka
- Dekompozicija protočne obrade
  - „fabričke trake" za podatke
  - Lanci proizvođača-potrošača

Uputstvo za dekompoziciju zadataka (1/2)

- Algoritmi nastaju iz dobrog razumevanja problema koji se rešava
- Programi se obično prirodno dekomponuju na zadatke
  - Dve česte dekompozicije su:
    - Pozivi funkcija
    - Različite iteracije u petlji
- Lakše je započeti sa više zadataka i kasnije ih spajati, nego sa manje zadataka koje kasnije treba razdvajati

Uputstvo za dekompoziciju zadataka (2/2)

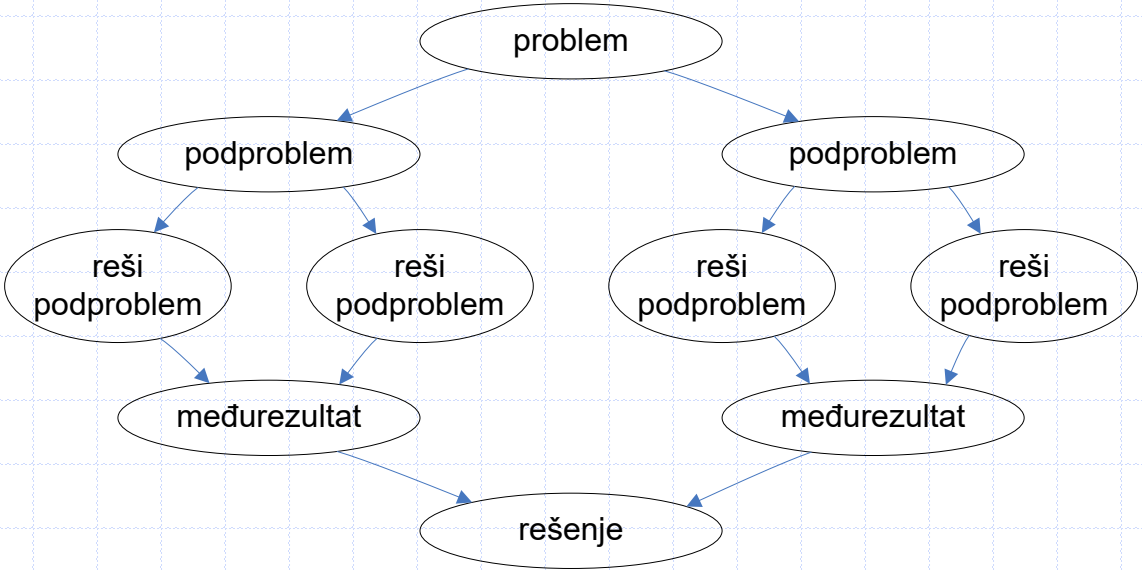
- Fleksibilnost u broju i veličini zadataka
  - Zadaci ne treba da budu vezani za specifičnu arhitekturu
  - Fiksni zadaci naspram parametrizovanih zadataka
- Efikasnost
  - Zadaci sa dovoljno posla da se amortizuje cena njihovog stvaranja i rukovanja
  - Zadaci treba da budu dovoljno nezavisni, kako rukovanje zavisnostima ne bi bilo usko grlo
- Jednostavnost
  - Kod mora ostati čitljiv i lak za razumevanje i održavanje (debagovanje)

Uputstvo za dekompoziciju podataka

- Dekompozicija podataka je obično određena dekompozicijom zadataka
- Programer mora adresirati ove dve dekompozicije da bi napravio paralelni program
  - Sa kojom započeti?
- Dekompozicija podataka je dobra početna tačka kada je:
  - Glavna obrada organizovana oko manipulacije velike strukture podataka
  - Primenjuju se slične operacije nad različitim delovima strukture podataka

Česte dekompozicije podataka

- Strukture podataka u obliku nizova
  - Dekompozicija nizova po vrstama, kolonama, blokovima
- Rekurzivne strukture podataka
  - Na primer: dekompozicija stabla u podstabla



Uputstvo za dekompoziciju podataka

- Fleksibilnost
  - Broj i veličina blokova podataka treba da podrže širok opseg izvršenja (npr. broj iteracija nije isti pri svakom pokretanju, tj. izvršavanju, programa)
- Efikasnost
  - Blokovi podataka treba da generišu uporedive količine posla (radi balansiranja opterećenja) – to obično znači da sami blokovi podataka treba da budu slične veličine
- Jednostavnost
  - Složene dekompozicije podataka mogu biti teške za obradu i otkrivanje grešaka

Slučaj za dekompoziciju protočne obrade

- Podaci teku kroz niz stepeni obrade
  - Dobra analogija je fabrička traka
- Glavni primer u računarskim arhitekturama?
  - Protočna obrada instrukcija u modernim procesorima (tzv. pajplajn)
- Primer u komandnom procesoru OS Unix?
  - Cevi (pipes) u Unix: cat foobar.c | grep bar | wc
- Drugi primeri
  - Obrada signala
  - Grafika

Analiza zavisnosti

- Ako su data dva zadatka, kako odrediti da li se oni mogu bezbedno izvršavati u paraleli?

Bernštajnov uslov

- R_i skup mem. lokacija koje čita zadatak T_i (ulaz)
- W_j skup lokacija u koje piše zadatak T_j (izlaz)

- Dva zadatka, T_1 i T_2, mogu biti paralelna ako:
  - Ulaz T_1 nije deo izlaza T_2
  - Ulaz T_2 nije deo izlaza T_1
  - Izlazi T_1 i T_2 se ne preklapaju

Primer: Da li ova dva zadatka mogu biti paralelna?

T1

a = x + y

T2

b = x + z

Primer: Da li ova dva zadatka mogu biti paralelna?

T1	T2
a = x + y	b = x + z

$R_1=\{x,y\}$, $W_1=\{a\}$, $R_2=\{x,z\}$, $W_2=\{b\}$

$R_1 \cap W_2 = \{x,y\} \cap \{b\} = \emptyset$

$R_2 \cap W_1 = \{x,z\} \cap \{a\} = \emptyset$

$W_1 \cap W_2 = \{b\} \cap \{a\} = \emptyset$

Četiri projektantska prostora

- Izražavanje algoritma
  - I Pronalaženje paralelizma
    - Uočavanje konkurentnih zadataka
  - II Struktura algoritma
    - Preslikavanje zadataka na procese radi korišćenja paralelnih arhitektura
- Konstruisanje programa
  - III Pomoćne strukture
    - Šabloni koda i struktura podataka
  - IV Izvedbeni mehanizmi
    - Mehanizmi niskog nivoa, koji se koriste za pisanje paralelnih programa

Projektantski prostor: Struktura algoritma

- Imamo određen skup paralelnih zadataka, šta je sledeće?
- Sledeći korak je preslikavanje zadataka na jedinice izvršenja (npr. procesi/niti)
- Važno je razmotriti
  - Broj jedinica izvršenja koje podržava ciljna platforma
  - Cenu deljenja informacije između jedinica izvršenja
  - Izbegavati suvišno ograničavanje implementacije
    - Treba da radi dobro na ciljnoj platformi
    - Ali da bude dovoljno fleksibilno, da se lako adaptira za različite arhitekture

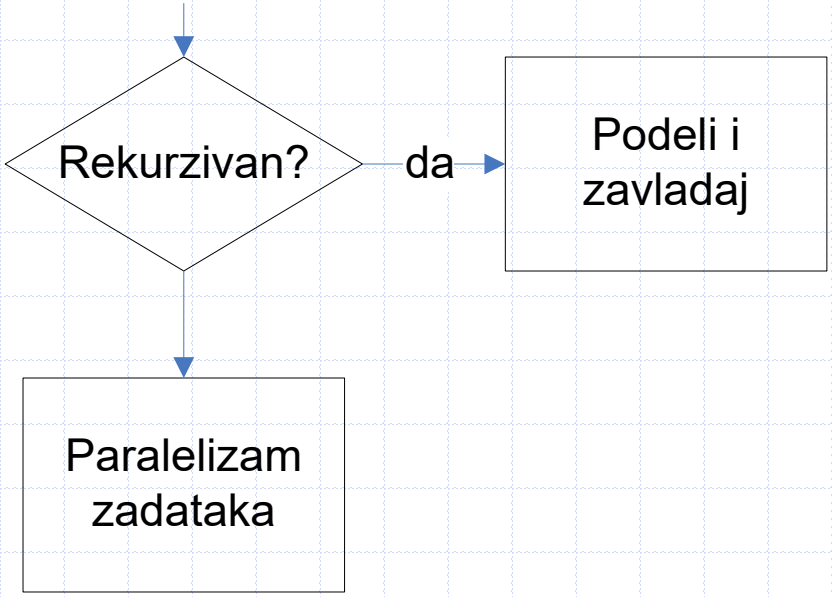
Glavni princip organizovanja

- Kako odrediti strukturu algoritma koja predstavlja preslikavanje zadataka na jedinice izvršenja?

- Paralelizam obično određuje glavni princip organizovanja
  - Organizovanje po zadacima
  - Organizovanje po dekompoziciji podataka
  - Organizovanje po toku podataka

Organizacija po zadacima

- Rekurzivni program => podeli i zavladaj
- Nerekurzivni program => paralelizam zadataka

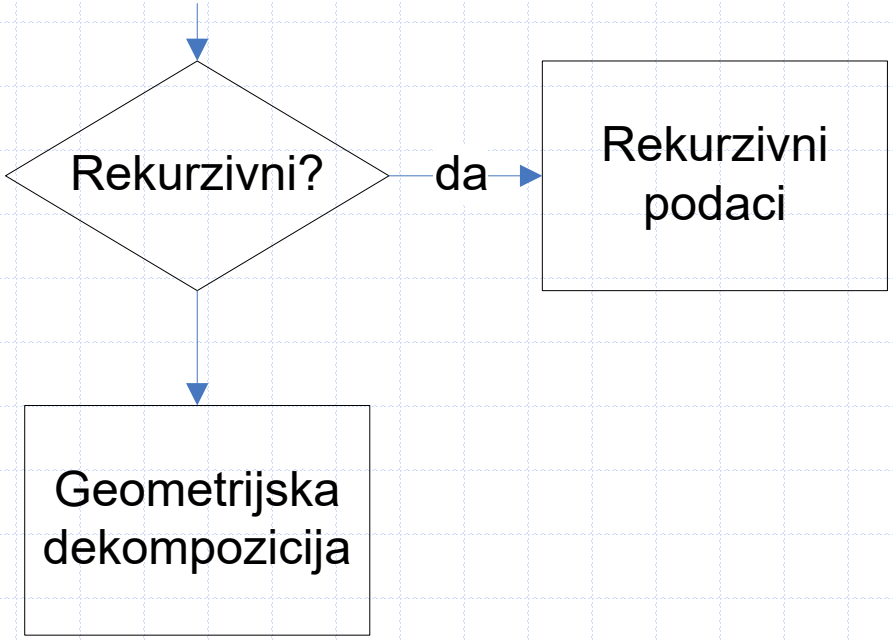


Paralelizam zadataka

- Praćenje zraka (ray tracing)
  - Obrada za svaki zrak je odvojena i nezavisna
- Dinamika molekula
  - Obrada među-atomskih sila, neke zavisnosti
- Zajednički činioci
  - Zadaci su pridruženi iteracijama petlje
  - Zadaci su uglavnom poznati na početku obrade
  - Ne moraju se svi zadaci završiti da bi se došlo do konačnog rešenja

Organizovanje po podacima

- Operacije na centralnoj strukturi podataka
  - Nizovi i linearne strukture podataka
  - Rekurzivne strukture podataka

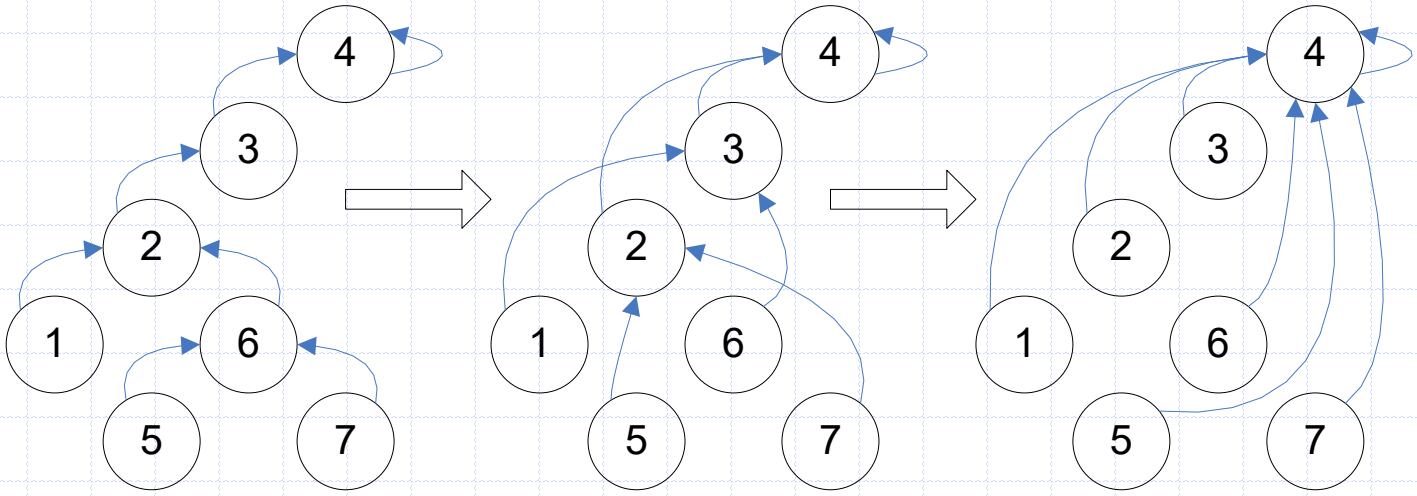


Rekurzivni podaci

- Obrade nad listama, stablima ili grafovima
  - Obično se pokazuje da je jedini način da se reši problem, da se sekvencijalno ide kroz struktu podataka
- Ali, pojavljuju se i prilike da se operacije preoblikuju tako da se ostvari paralelizam

Primer rekurzivnih podataka: Pronalaženje korena

- Ako je data šuma usmerenih stabala sa korenom, za svaki čvor pronađi koren njegovog stabla
  - Paralelni pristup: za svaki čvor pronađi prethodnikovog prethodnika, ponavljaj dok ima izmena
    - O(log n) naspram O(n)



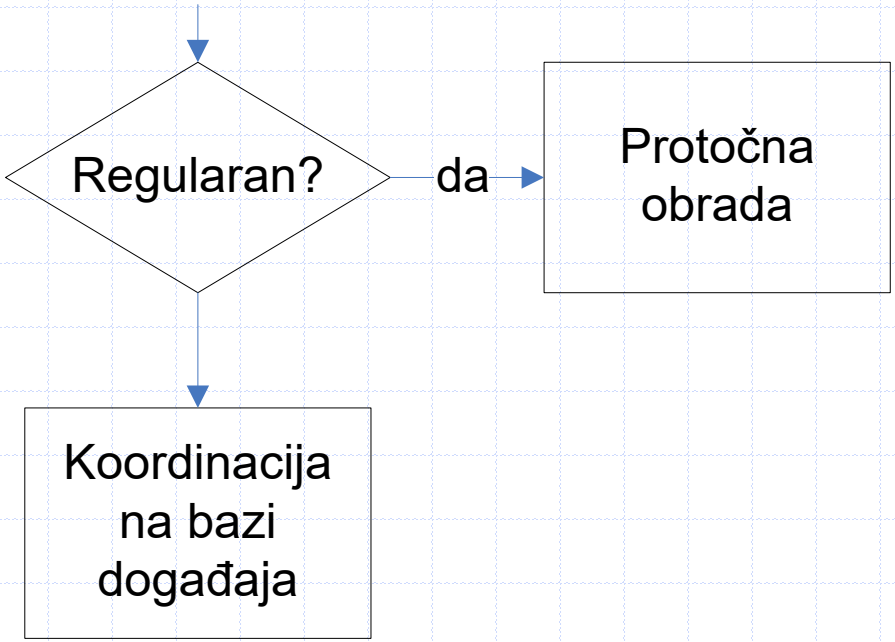
Kompromis: Količina posla naspram paralelizma

- Paralelno prestruktuiranje algoritma za pronalaženje korena dovodi do O(n log n) naspram O(n) za sekvencijalan pristup, ali O(log n) za paralelni pristup

- Većina strategija zasnovanih na ovom šablonu dopuštaju povećanje ukupne količine posla radi skraćenja vremena izvršenja na osnovu iskorišćenog paralelizma

Organizacija po toku podataka

- U nekim aplikacionim domenima, tok podataka nameće redosled zadataka
  - Regularan, u jednom smeru, uglavnom stabilan tok
  - Iregularan, dinamički, nepredvidivi tok podataka



Propusnost protočne obrade naspram kašnjenja

- Paralelizam protočne obrade je ograničen brojem stepeni protočne obrade
- Radi dobro ako je vreme punjenja i pražnjenja protočne obrade malo u odnosu na vreme obrade
- Mera performanse je obično propusnost
  - Stopa kojom se podaci pojavljuju na kraju protočne obrade (npr. broj okvira u sekundi)
- Kašnjenje protočne obrade je važno za aplikacije u realnom vremenu
  - Vremenski interval od ulaza podataka u protočnu obradu do izlaza

Koordinacija zasnovana na događajima

- U ovom šablonu interakcija zadataka radi obrade podataka se dešava u nepredvidivim intervalima vremena
- U aplikacijama koje koriste ovaj šablon moguća su međusobna blokiranja zadataka (deadlock)